Linux for Beginners

A Practical Guide

Chapter 6 of 12

Text Editors: Reading and Editing Files in the Terminal

Contents

Chapter 6

Text Editors: Reading and Editing Files in the Terminal

6.1 Why Terminal Text Editors Matter

When you're working on a remote server over SSH, there's no graphical interface. No VS Code window, no file manager, no desktop. Just a terminal. In that environment — which describes the majority of real Linux work in the professional world — you need to be able to open, read, and edit files without leaving the command line.

Even on a local machine, terminal editors are often faster than launching a graphical app. Editing a configuration file, fixing a script, or checking a log entry mid-workflow takes two seconds in the terminal and interrupts nothing.

This chapter covers two editors: **nano** (simple and approachable — the right starting point) and **vim** (powerful and ubiquitous — worth learning properly). Both are installed on almost every Linux system in existence.

6.2 A Map of the Landscape: Choosing an Editor

Linux has several terminal text editors, each with a different philosophy. Here's where they sit relative to each other:

Editor	Learning Curve	Power	Best For
nano	Very gentle	Basic	Quick edits, beginners, config files
vim	Steep initially	Very high	Power users, remote work, daily editing
neovim	Steep	Very high	Developers wanting a modern vim
emacs	Very steep	Extreme	Enthusiasts who want an entire ecosystem
micro	Gentle	Moderate	nano users who want more features
ed	Arcane	Low	Historical; scripting line edits

> **TIP**
> Start with nano. Learn it until editing files feels natural. Then invest time in vim — it will pay back that investment every day for the rest of your Linux life.

6.3 nano — The Beginner-Friendly Editor

nano is designed around one principle: it should be immediately usable without reading a manual. When you open nano, all the commands you need are displayed at the bottom of the screen. The ^ symbol means Ctrl.

```bash
# Open a file (creates it if it doesn't exist):
$ nano notes.txt

# Open with the cursor on a specific line:
$ nano +42 script.sh

# Open a system config file (with sudo):
$ sudo nano /etc/hosts
```

What you see when nano opens:

```nano
  GNU nano 6.2          notes.txt

Hello, this is my notes file.
I can type here freely.




^G Help    ^O Write Out  ^W Where Is  ^K Cut
^X Exit    ^R Read File  ^\ Replace   ^U Paste
```

The bottom two rows show the most common shortcuts. ^ means Ctrl — so ^X is Ctrl+X (exit), ^0 is Ctrl+O (write/save).

6.4 nano in Depth: Essential Operations

Saving and exiting

```nano
Ctrl + O    Write (save) the file — nano will confirm the filename
Ctrl + X    Exit nano
            → if unsaved changes: nano asks Save? [Y/n]
            → press Y then Enter to confirm the filename and save
            → press N to discard changes
            → press Ctrl+C to cancel and stay in nano
```

Navigation

Shortcut	Action
Arrow keys	Move cursor one character / line at a time
Ctrl + A	Jump to start of current line
Ctrl + E	Jump to end of current line
Ctrl + Y	Scroll up one page
Ctrl + V	Scroll down one page
Ctrl + _	Go to a specific line number (prompts you)
Alt + G	Go to a specific line (alternative)
Ctrl + W	Search — type a term and press Enter
Alt + W	Find next occurrence of the search term

Editing

Shortcut	Action
Ctrl + K	Cut the current line (into nano's clipboard)
Ctrl + U	Paste the cut line
Alt + 6	Copy the current line without cutting
Ctrl + \	Find and replace — prompts for search term then replacement
Alt + U	Undo last action
Alt + E	Redo
Ctrl + D	Delete the character under the cursor
Ctrl + H	Delete the character before the cursor (backspace)

> **TIP**
> The most important nano sequence to memorise: Ctrl+O (save), Ctrl+X (exit).
> Everything else you can look up in the built-in help: Ctrl+G.

6.5 vim — The Powerful Modal Editor

`vim` (Vi IMproved) is one of the most widely used text editors in the world. It's available on virtually every Unix and Linux system. Sysadmins, developers, and DevOps engineers use it daily. It is also the editor that has inspired the most memes about people being unable to exit it.

vim is genuinely harder to start with than nano. The learning curve is front-loaded and steep. But users who invest time in vim consistently report that it becomes the fastest editing environment they've ever used — because once the commands are in muscle memory, you rarely need to reach for the mouse.

```bash
# Open a file:
$ vim notes.txt

# Open vim without a file:
$ vim

# Open multiple files:
$ vim file1.txt file2.txt

# Check your vim version:
$ vim --version | head -1
```

6.6 vim's Modal System: The Key Insight

The single most important thing to understand about vim is that it is a **modal editor**. This means the keyboard behaves differently depending on which **mode** you're in. Most editors have only one mode — you type and text appears. vim has several.

Mode	How to Enter	What keys do
Normal	Esc (from any mode)	Commands: move cursor, delete, copy, paste, search
Insert	i (before cursor) or a (after)	Type text — like a normal editor
Visual	v (char), V (line), Ctrl+V (block)	Select text for copy/cut/replace operations

Mode	How to Enter	What keys do
Command	: (colon)	Run editor commands: save, quit, find-replace, settings
Replace	R	Overwrite existing text character by character

vim **always starts in Normal mode**. This is the mode for navigating and issuing commands — not for typing text. New users instinctively start typing and get confused when nothing expected happens. The fix: press `i` to enter Insert mode first.

> **TIP**
> The golden rule of vim: when in doubt, press Esc. Pressing Esc always returns you to Normal mode, no matter what mode you're currently in. It is your reset button.

6.7 vim in Depth: Navigation, Editing, and Saving

The essential workflow

```vim
# 1. Open a file:
$ vim notes.txt

# 2. You're in Normal mode. Navigate to where you want to edit.

# 3. Press i to enter Insert mode:
i

# 4. Type your text:
This is new text I'm adding.

# 5. Press Esc to return to Normal mode:
Esc

# 6. Save and quit:
:wq
```

Navigation in Normal mode

vim's navigation keys are designed to keep your fingers on the home row:

Key(s)	Action
h / j / k / l	Left / Down / Up / Right (arrow keys also work)

Key(s)	Action
w	Jump forward to start of next word
b	Jump backward to start of previous word
e	Jump to end of current word
0	Jump to start of line
$	Jump to end of line
gg	Jump to first line of file
G	Jump to last line of file
42G	Jump to line 42
Ctrl + F	Scroll forward (down) one page
Ctrl + B	Scroll backward (up) one page
%	Jump to matching bracket: ( ), [ ], { }

Editing in Normal mode (no Insert mode needed)

Key(s)	Action
x	Delete character under cursor
dd	Delete (cut) entire current line
5dd	Delete 5 lines
yy	Yank (copy) current line
5yy	Yank 5 lines
p	Paste after cursor / below current line
P	Paste before cursor / above current line
u	Undo last action
Ctrl + R	Redo
.	Repeat the last change — extremely powerful
dw	Delete from cursor to end of word
cw	Change word (delete and enter Insert mode)

Key(s)	Action
cc	Change entire line (delete and enter Insert mode)
A	Append to end of line (enters Insert mode)
o	Open new line below and enter Insert mode
O	Open new line above and enter Insert mode

Saving and quitting (Command mode)

```vim
:w          save (write) the file
:q          quit (fails if unsaved changes)
:wq         save and quit
:x          save and quit (only writes if changes exist)
:q!         quit WITHOUT saving — discard all changes
:w filename save to a different filename
:wqa        save and quit all open files
```

6.8 vim: Search, Replace, and Multiple Files

Searching

```vim
# In Normal mode, press / to search forward:
/searchterm     search forward
?searchterm     search backward
n               jump to next match
N               jump to previous match
*               search for the word under the cursor
```

Find and replace

vim's substitution command is one of its most powerful features, using a syntax similar to the `sed` tool:

```vim
:s/old/new          replace first match on current line
:s/old/new/g        replace all matches on current line
:%s/old/new/g       replace all matches in entire file
:%s/old/new/gc      replace all, ask for confirmation each time
:10,20s/old/new/g   replace all matches between lines 10 and 20
```

Working with multiple files

```vim
$ vim file1.txt file2.txt   # open both files
:n                          # switch to next file
:N                          # switch to previous file
:e filename                 # open another file
:split filename             # horizontal split
:vsplit filename            # vertical split
Ctrl + W then W             # switch between split panes
```

6.9 Configuring vim with .vimrc

vim's behaviour is controlled by a configuration file in your home directory: `~/.vimrc`. It doesn't exist by default — you create it. Every line in `.vimrc` is a vim command that runs automatically when vim starts.

```vim
$ vim ~/.vimrc

" A sensible starter .vimrc (lines starting with " are comments)
set number          " show line numbers
set relativenumber  " show relative line numbers (great for navigation)
set tabstop=4       " tab = 4 spaces
set shiftwidth=4    " indent = 4 spaces
set expandtab       " use spaces instead of tabs
set autoindent      " keep indentation on new lines
set hlsearch        " highlight search results
set incsearch       " search as you type
set ignorecase      " case-insensitive search
set smartcase       " ...unless you use uppercase letters
set wrap            " wrap long lines
set ruler           " show cursor position in status bar
set showmatch       " highlight matching brackets
syntax on           " enable syntax highlighting
colorscheme desert  " set a colour scheme
```

> **TIP**
> After editing .vimrc, reload it without restarting vim: type :source ~/.vimrc in Command mode. Changes take effect immediately.

6.10 Editing System Configuration Files Safely

Many Linux administration tasks involve editing system config files — files owned by root in directories like `/etc/`. These files control critical system behaviour, so editing them carelessly can break things. Here's how to do it safely.

Always use sudo

```bash
$ sudo nano /etc/hosts           # edit with nano
$ sudo vim /etc/ssh/sshd_config  # edit with vim
```

Make a backup before editing

```bash
# Always back up before editing critical config files:
$ sudo cp /etc/hosts /etc/hosts.bak
$ sudo nano /etc/hosts

# If something breaks, restore from backup:
$ sudo cp /etc/hosts.bak /etc/hosts
```

Common config files you'll edit

File	What it controls
/etc/hosts	Local hostname to IP mappings — useful for blocking sites or local dev
/etc/fstab	Filesystem mount points — which drives mount where at boot
/etc/ssh/sshd_config	SSH server settings — port, auth methods, access rules
/etc/crontab	System-wide scheduled tasks
~/.bashrc	Your personal shell configuration — aliases, environment variables
~/.bash_profile	Login shell configuration — runs once when you log in

> **WARNING**
> Never edit /etc/fstab, /etc/sudoers, or SSH config files without a backup. A mistake in fstab can prevent your system from booting; a mistake in sshd_config can lock you out of a remote server. Always: copy first, edit second.

6.11 Viewing Files Without Editing: A Refresher

Sometimes you just want to read a file — not edit it. Using a full editor for this is overkill. Here's a quick reminder of the right tools, covered in depth in Chapter 3:

Command	Best For	Key Feature
cat file	Small files, quick read	Dumps entire file instantly

Command	Best For	Key Feature
less file	Large files, comfortable reading	Scroll, search with /, quit with q
head -n N file	Check start of file	Shows first N lines (default 10)
tail -n N file	Check end of file / logs	-f flag follows file live
grep 'term' file	Find lines containing a pattern	grep -i for case-insensitive search
wc -l file	Count lines in a file	-w for words, -c for characters

6.12 nano vs vim: Which Should You Learn?

The honest answer: learn both, but start with nano and add vim over time.

Question	nano	vim
Can I use it immediately?	Yes	No — needs practice
Available on every server?	Often, not always	Yes — always present
Good for quick config edits?	Yes	Yes (once learned)
Good for coding / long sessions?	No	Yes
Mouse support?	Yes	Optional
Keyboard shortcuts on screen?	Yes — bottom of screen	No — must memorise
Extensible / scriptable?	Limited	Extremely
Used by professionals daily?	Some	Very widely

A practical learning path: use **nano for the next month** for all your file editing. Once it feels natural, spend 30 minutes with `vimtutor` (a built-in interactive tutorial — just type `vimtutor` in the terminal). Then switch to vim for small tasks and build from there.

6.13 Essential Commands: Quick Reference

nano

Shortcut	Action
nano file	Open file in nano

Shortcut	Action
Ctrl + O	Save (write out) the file
Ctrl + X	Exit nano
Ctrl + W	Search
Ctrl + \	Find and replace
Ctrl + K	Cut current line
Ctrl + U	Paste
Ctrl + G	Open help
Alt + U	Undo
Ctrl + _	Go to line number

vim

Key / Command	Mode	Action
vim file	—	Open file in vim
i	Normal	Enter Insert mode before cursor
a	Normal	Enter Insert mode after cursor
o / O	Normal	Open new line below / above, enter Insert
Esc	Any	Return to Normal mode
h j k l	Normal	Move left / down / up / right
gg / G	Normal	Jump to first / last line
dd	Normal	Cut (delete) current line
yy	Normal	Copy (yank) current line
p	Normal	Paste below cursor
u / Ctrl+R	Normal	Undo / Redo
.	Normal	Repeat last change
/term	Normal	Search forward for term
n / N	Normal	Next / previous search match

Key / Command	Mode	Action
:%s/old/new/g	Command	Replace all occurrences in file
:w	Command	Save the file
:q!	Command	Quit without saving
:wq	Command	Save and quit
vimtutor	—	Built-in interactive vim tutorial

6.14 Chapter Summary

✓ Terminal text editors are essential for remote server work and fast on-the-spot file editing — no GUI required.

✓ **nano** is the right starting point: simple, approachable, and displays its own shortcuts at the bottom of the screen.

✓ **vim** is a modal editor. It starts in Normal mode. Press `i` to type, `Esc` to return to Normal. Press `Esc` whenever lost.

✓ The key vim commands to memorise first: `i` (insert), `Esc` (normal), `:wq` (save and quit), `:q!` (quit without saving).

✓ vim navigation: `gg` (top), `G` (bottom), `dd` (cut line), `yy` (copy line), `p` (paste), `u` (undo).

✓ vim search: `/term` to search, `n`/`N` to navigate. Replace with `:%s/old/new/g`.

✓ `~/.vimrc` configures vim. Add line numbers (`set number`), syntax highlighting (`syntax on`), and tab settings at minimum.

✓ Always **back up** system config files before editing them with `sudo`. A mistake in `/etc/fstab` or `sshd_config` can have serious consequences.

✓ Use `vimtutor` — the built-in interactive tutorial — to learn vim properly. It takes about 30 minutes and covers everything you need to get started.

What's Coming in Chapter 7

With file editing covered, Chapter 7 turns to what's happening *right now* on your system. You'll learn how to see and control running processes:

- What a process is — and how Linux identifies every running program with a PID

- Viewing processes with `ps`, `top`, and `htop`
- Sending signals to processes: `kill`, `killall`, `pkill`
- Running processes in the background with `&`, `bg`, `fg`, and `nohup`
- Monitoring system resources: CPU, memory, disk I/O
- Understanding load average and when your system is under stress

Chapter 7 is where you learn to see inside a running Linux system.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.